

Installation guide - Multiranger deck first, AI deck second approach

material requirements

- a room you want to map out (minimum 2m by 2m)
- 4 crazyflie drones with flow decks on the bottom of each drone
 - two drones have an AI deck on top of them with the [correct radio addresses](#)
 - two drones have a ranger deck on top of them with the [correct radio addresses](#)
- 1 Crazyradio 2.0
- a laptop or PC

Drone setup

Crazyradio

Plug the Crazyradio 2.0 into your laptop or pc

Drone addresses and channels

Before running the mission, every drone needs to have the correct radio address. This can be checked and changed with the Crazyflie Client.

First open the Crazyflie Client and connect one drone at a time. It is best to only turn on one drone while changing its address, so you do not accidentally change the wrong drone.

The drones should use these addresses:

```
M stands for multiranger deck equiped drone
AI stands for AI deck equiped drone
```

```
M1  = radio://0/80/2M/E7E7E7E701
M2  = radio://0/80/2M/E7E7E7E703
AI1 = radio://0/84/2M/E7E7E7E702
AI2 = radio://0/84/2M/E7E7E7E704
```

M1 and M2 use channel **80**. AI1 and AI2 use channel **84**. All drones use **2M** datarate. The last part of the address is different for every drone, so the program knows which drone is which.

To change the address, open the Crazyflie Client, connect to the drone, and go to the configuration page where the radio address can be changed. After changing the address, write the setting to the drone and restart it. Do this for every drone until all four drones have the correct address.

A good way to avoid mistakes is to write a small label on each drone: **M1**, **M2**, **AI1**, and **AI2**.

Front side of the drone

For this project, the side with the start/power button is used as the front side of the drone. When the guide says that a drone has to face a certain direction, this means that the start/power button side should point in that direction.

This is important for the placement. M1 and M2 have to face opposite directions, but AI1 and AI2 both have to face the same direction as M1.

Charging and turning on the drones

Before testing, make sure all four drones are fully charged. To charge a drone, turn it off and plug it in with a USB cable. Wait until it is charged before flying. A weak battery can cause a drone to drop during the mission or reboot in the air.

When the drones are charged, place them in the correct positions while they are still turned off. After all drones are placed correctly, turn them on one by one. Wait until each drone has started properly before running the code.

Drone placement

Before placing the drones, make sure all drones are turned off.

Use the middle line of the room as the reference line.

Multiranger drones

Place the two Multiranger drones close to the middle line, with a total distance of about 30 cm between them.

Place M1 about 15 cm on one side of the middle line. Place M2 about 15 cm on the other side of the middle line. M1 must have the wall on its left side. M2 must have the wall on its right side. M1 and M2 should face opposite directions.

The Multiranger drones will first map the room and create the safe zone.

AI-deck drones

Place the two AI-deck drones further away from the middle line.

Place AI1 about 75 cm in front of the middle line. Place AI2 about 75 cm behind the middle line. AI1 and AI2 must both face the same direction as M1.

The AI drones will launch after the mapping is finished. AI1 scans one half of the safe zone, and AI2 scans the other half. If an obstacle is detected in the map, the AI drone responsible for that half will fly to safe viewpoints around the obstacle.

Dependencies

In the source folder you have the following sub-folders

- `Fully_drone_movement/crazyflie_wall_following_pointcloud` or `Fully_drone_movement\crazyflie_wall_following_pointcloud` on Windows
- `image_stream`

- `map_api_adaptation`

With each sub folder as current directory you must execute the following commands

to create a virtual environment

```
python3.12 -m venv .venv # create a virtual environment
```

```
source .venv/bin/activate # to enter the virtual environment... for Windows  
use: .venv\Scripts\activate
```

```
pip install -r requirements.txt # to install all required dependencies to  
run the apps
```

```
deactivate # exit the virtual environment before moving onto the next  
folder
```

Running the apps

Each application must be run in a separate terminal session.

Map Api and Interface

from the `map_api_adaptation` folder

```
source .venv/bin/activate # enter the virtual environment... for Windows  
use: .venv\Scripts\activate
```

```
docker compose up -d --build # build the docker compose project (api and  
database)
```

```
python app_api_gradio.py # run the app on http://127.0.0.1:7860
```

Wall following and Safezones

from the `Fully_drone_movement/crazyflye_wall_following_pointcloud` folder or
`Fully_drone_movement\crazyflye_wall_following_pointcloud` folder on Windows

```
source .venv/bin/activate # enter the virtual environment... for Windows  
use: .venv\Scripts\activate
```

Starting the drones

With all drones are placed correctly, turn them on.

Running the mission

```
python main_v10_corners.py # run the 2D mapping mission
```

press ENTER for the mission to start

The mission will run in this order.

First, M1 and M2 connect. Then M1 and M2 take off and map the room. After the mapping is finished, a merged safe-zone map is created. If an obstacle is detected, it is exported to the safe-zone file.

After this, AI1 and AI2 connect. They both wait on the ground until they are ready. Then AI1 and AI2 launch together. AI1 scans one half of the safe zone and AI2 scans the other half. If an obstacle is present, the correct AI drone looks at it from safe viewpoints. When the AI drones are finished, they return to their own start position and land.

Safety controls

During the mission, the keyboard can be used for safety.

```
L or SPACE = smooth land  
E           = emergency stop  
Ctrl + C   = stops the program (secondary emergency stop)
```

Use these if something goes wrong during flight.

Important notes

Do not move the drones after turning them on and starting the script.

Make sure the drones are fully charged before testing. A weak battery can cause a drone to drop during flight or reboot.

The `safe_zone_output` and `mission_control` folders are generated during the mission. They do not need to contain old files before starting a new run. Old files in these folders can be deleted, because new output is generated every run.

Image stream

Connect to the AI deck wifi hotspot of your choice.

```
source .venv/bin/activate # enter the virtual environment... for Windows  
use: .venv\Scripts\activate
```

```
python app.py # run the app
```

Installation guide - Grouped approach

Project

This guide explains how to install, run, test, and present the Ranger-guided AI-deck streaming swarm project.

The guide is folder-independent. Replace the placeholders below with paths from your own computer.

Path placeholders

<PROJECT_ROOT>	Folder where the main project is located
<DRONE_CONNECTION_DIR>	<PROJECT_ROOT>/testing/drone_connection
<VENV_DIR>	<PROJECT_ROOT>/.venv
<AI_DECK_IP>	IP address of an AI-deck stream server

<PROJECT_ROOT> must be the folder that contains the `testing` directory.

Example:

```
<PROJECT_ROOT>/testing/drone_connection/gui/main.py
```

means: open your own project folder, then go to `testing/drone_connection/gui/main.py`.

On Windows, PowerShell usually accepts both `/` and `\` in paths. In this guide, project-relative paths use `/` to avoid duplicating the same command for every operating system.

Main GUI entry point:

```
testing/drone_connection/gui/main.py
```

Main backend controller:

```
testing/drone_connection/swarm_controller/controller.py
```

1. Hardware Setup

For the current half-swarm presentation, use three drones:

Role	Radio URI	Decks	Purpose
X_FRONT	radio://0/82/2M/E7E7E7E701	Ranger + Flow	Scout and safety leader
01	radio://0/82/2M/E7E7E7E702	AI deck + Flow	Left visual observer
02	radio://0/82/2M/E7E7E7E703	AI deck + Flow	Forward/right visual observer

Use the launch mat coordinates:

```
X_FRONT: x=32, y=20
01:      x=12, y=45
02:      x=52, y=45
```

All drones should physically face the same forward direction before takeoff.

After takeoff, the controller aligns camera yaw:

```
X_FRONT -> movement heading
01      -> 270 deg, left-looking stream
02      -> 0 deg, forward-looking stream
```

2. Wi-Fi Setup

The AI decks should connect to the existing Wi-Fi network:

```
SSID: swarming
Password: swarming
AI-deck stream port: 5000
```

The laptop should also be connected to **swarming** when using live AI-deck previews.

AI-deck streaming code:

```
testing/drone_connection/src/aideck_stream.py
testing/drone_connection/gui/ai_stream_preview.py
```

```
testing/drone_connection/gui/widgets/ai_stream_panel.py
testing/drone_connection/aideck-gap8-examples/examples/other/wifi-img-
streamer/wifi-img-streamer.c
```

3. Install Python Environment

3.1. Open a terminal

Use:

- **Windows:** PowerShell
- **macOS/Linux:** Terminal

3.2. Go to the project root

```
cd "<PROJECT_ROOT>"
```

3.3. Create a virtual environment

Use the command for your operating system:

Windows PowerShell

```
py -3 -m venv .venv
```

macOS/Linux

```
python3 -m venv .venv
```

3.4. Activate the virtual environment

Use the command for your operating system:

Windows PowerShell

```
& ".\venv\Scripts\Activate.ps1"
```

If PowerShell blocks activation, run this once for the current terminal session:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass  
& ".\venv\Scripts\Activate.ps1"
```

macOS/Linux

```
source .venv/bin/activate
```

After activation, the terminal should show something like:

```
(.venv)
```

3.5. Install dependencies

From `<PROJECT_ROOT>`, run:

```
python -m pip install --upgrade pip  
pip install -r testing/drone_connection/requirements.txt
```

The requirements file is:

```
testing/drone_connection/requirements.txt
```

It installs the main project dependencies, including:

```
numpy  
opencv-python  
cflib  
cfclient  
h5py  
PyQt6
```

4. Verify Installation

Go to the drone connection directory:

```
cd "<PROJECT_ROOT>/testing/drone_connection"
```


Verify Python dependencies:

```
python -c "import PyQt6, cflib, cv2, numpy; print('installation OK')"
```

Check Crazyradio / Crazyflie interfaces:

```
python -c "import cflib.crtp; cflib.crtp.init_drivers();  
print(cflib.crtp.scan_interfaces())"
```

Expected result when Crazyradio and/or USB Crazyflie are visible:

```
[('radio://...', '...')]
```

or:

```
[('usb://0', '')]
```

If the result is empty, reconnect Crazyradio, check drivers, close other Crazyflie programs, and try again.

5. Run The Final Swarm GUI

The GUI is the preferred way to run the project during testing and presentation.

Make sure the virtual environment is activated, then run:

```
cd "<PROJECT_ROOT>/testing/drone_connection"  
python -m gui.main
```

Main GUI files:

```
testing/drone_connection/gui/main.py  
testing/drone_connection/gui/main_window.py  
testing/drone_connection/gui/worker.py
```

GUI widget files:

```
testing/drone_connection/gui/widgets/drone_status_panel.py
testing/drone_connection/gui/widgets/mission_control_panel.py
testing/drone_connection/gui/widgets/ranger_panel.py
testing/drone_connection/gui/widgets/formation_view.py
testing/drone_connection/gui/widgets/ai_stream_panel.py
testing/drone_connection/gui/widgets/event_log_panel.py
testing/drone_connection/gui/widgets/thresholds_panel.py
```

6. GUI Usage Workflow

For a presentation or real test, use this order:

1. Place the drones on the launch mat.
2. Connect Crazyradio to the laptop.
3. Turn on the drones.
4. Connect the laptop to Wi-Fi **swarming**.
5. Start the GUI.
6. Click **Load Half Group (3)**.
7. Click **Use Real Crazyflies**.
8. Click **Connect All**.
9. Click **Sensor Check**.
10. Open the **AI Streams** tab.
11. Click **Scan swarming Wi-Fi for AI decks**.
12. Confirm that O1/O2 IPs are filled or enter them manually.
13. Click **Start AI Streams**.
14. Click **Formation Hover Only** for a safe hover demonstration.
15. Click **Formation Micro Step** for a small movement proof.
16. Click **Start Full Observation Mode** for the full autonomous observation demo.
17. Use **SAFE HOVER / LAND** or **Land All** to finish.

Emergency buttons are always active:

```
EMERGENCY STOP / HARD KILL
SAFE HOVER / LAND
HARD MOTOR KILL
```

7. Simulation Mode

Simulation is useful before a real flight demonstration.

Start the GUI and click:

```
Run Simulation Mode
Sim Open
```

```
Sim Obstacle  
Sim Wall  
Start Top-3 Observation Demo
```

Command-line simulation from `<DRONE_CONNECTION_DIR>`:

```
cd "<PROJECT_ROOT>/testing/drone_connection"  
python -m swarm_controller.main --simulate --half-group --scenario  
open_space --steps 5
```

Other useful scenarios:

```
python -m swarm_controller.main --simulate --half-group --scenario obstacle  
--steps 5  
python -m swarm_controller.main --simulate --half-group --scenario wall --  
steps 5
```

Simulation code:

```
testing/drone_connection/swarm_controller/simulation_stub.py
```

8. Real Drone Backend Run

The GUI is the preferred presentation interface. The backend can also be run directly.

From `<DRONE_CONNECTION_DIR>`:

```
cd "<PROJECT_ROOT>/testing/drone_connection"  
python -m swarm_controller.main --half-group --mission-id  
final_demo_half_group --steps 5 --x-front-uri radio://0/82/2M/E7E7E7E701 --  
o1-uri radio://0/82/2M/E7E7E7E702 --o2-uri radio://0/82/2M/E7E7E7E703
```

Backend files:

```
testing/drone_connection/swarm_controller/main.py  
testing/drone_connection/swarm_controller/controller.py  
testing/drone_connection/swarm_controller/state_machine.py  
testing/drone_connection/swarm_controller/drones.py  
testing/drone_connection/swarm_controller/motion.py  
testing/drone_connection/swarm_controller/safety.py  
testing/drone_connection/swarm_controller/ranger.py
```

```
testing/drone_connection/swarm_controller/formation.py
testing/drone_connection/swarm_controller/logs.py
```

9. AI-Deck Stream Viewer

Standalone AI-deck stream GUI from `<DRONE_CONNECTION_DIR>`:

```
cd "<PROJECT_ROOT>/testing/drone_connection"
python main.py
```

Command-line stream viewer:

```
python main.py --cli --host <AI_DECK_IP> --port 5000 --max-fps 2
```

Save frames:

```
python main.py --cli --host <AI_DECK_IP> --port 5000 --save --max-fps 2
```

Saved frames go under:

```
testing/drone_connection/stream_out/
```

10. Mission Logs

Every mission writes evidence logs to:

```
testing/drone_connection/stream_out/fixed_formation_missions/<mission_id>_<timestamp>/
```

Important files:

```
README_LOGS.md
run_summary.json
formation_config.json
mission_log.csv
command_log.csv
ranger_log.csv
safety_grid_log.csv
```

```
capture_manifest.csv
classification_log.csv
reslot_path_log.csv
turn_reslot_log.csv
breadcrumb_log.csv
preflight_log.csv
battery_log.csv
mission_timing_log.csv
error_log.csv
```

Log implementation:

```
testing/drone_connection/swarm_controller/logs.py
```

11. Tests

Run the focused project tests from `<DRONE_CONNECTION_DIR>`:

```
cd "<PROJECT_ROOT>/testing/drone_connection"
python -m unittest gui.tests.test_button_wiring
swarm_controller.tests.test_simulation
swarm_controller.tests.test_controller_behavior
swarm_controller.tests.test_top3_logic
```

These tests check:

- GUI button wiring;
- simulation behavior;
- controller behavior;
- top-3 formation logic.

Expected result:

```
Ran 51 tests
OK
```

12. Presentation Run Checklist

Before the final presentation:

1. Charge all drone batteries.
2. Check propellers.
3. Confirm Crazyradio is connected.

4. Confirm laptop is connected to **swarming**.
5. Confirm drones are placed on the launch mat:

```
X_FRONT: x=32, y=20  
01:      x=12, y=45  
02:      x=52, y=45
```

6. Confirm all drones face the same physical forward direction.
7. Start the GUI.
8. Run **Load Half Group (3)**.
9. Run **Use Real Crazyflies**.
10. Run **Connect All**.
11. Run **Sensor Check**.
12. Start AI streams.
13. Run **Formation Hover Only**.
14. Run **Formation Micro Step**.
15. Run **Start Full Observation Mode**.
16. Finish with **SAFE HOVER / LAND** or **Land All**.

13. Troubleshooting

GUI does not start

Check PyQt6:

```
python -c "import PyQt6; print('PyQt6 OK')"
```

If PyQt6 is missing:

```
pip install PyQt6
```

Crazyradio not found

Check Crazyradio / Crazyflie interfaces:

```
python -c "import cflib.crtp; cflib.crtp.init_drivers();  
print(cflib.crtp.scan_interfaces())"
```

Reconnect Crazyradio, close other Crazyflie programs, then restart the GUI.

AI-deck stream IP unknown

Use the GUI:

```
AI Streams tab -> Scan swarming Wi-Fi for AI decks
```

or read the IP from Crazyflie Client console logs.

Flow/zrange warning

Use a matte floor with visible texture and good light. Flow decks need floor texture to stabilize correctly.

Mission behavior should be checked first

Use simulation before real flight:

```
Run Simulation Mode -> Sim Open -> Start Top-3 Observation Demo
```

14. Related Documentation

Presentation document:

```
testing/drone_connection/docs/final-presentation-documentation.md
```

System concept:

```
testing/drone_connection/docs/adaptive-360-mapping-swarm.md
```

Half-swarm behavior:

```
testing/drone_connection/docs/half-swarm-safe-mvp-logic.md
```

AI-deck streaming:

```
testing/drone_connection/docs/ai-deck-streaming.md
```

Backend README:

```
testing/drone_connection/swarm_controller/README.md
```